

SIMATS ENGINEERING

ASSESSMENT TOOL 3

**COURSE CODE: CSA1405
DESIGN**

COURSE NAME: COMPILER

COURSE OUTCOME COVERED

***CO3:** Analyze syntax-directed translation method using synthesized and inherited attributes to generate a Parser Tree. (BL4)*

Assessment Tool Weightage: 20%

QUESTIONS:

Total Marks:100

Annexure A

CASE STUDY

Code of Conduct:

*I **Lakshan Raj D/192521036** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

***Signature of the student:**
Lakshan Raj D/192521036*

Annexure A

CASE STUDY

Q1. Both parse trees and syntax trees are used to represent program structure, but differ significantly in complexity and abstraction level. Compare these representations in terms of memory usage, ease of traversal, suitability for optimization, and their role in different phases of compilation such as semantic analysis and code generation. These require careful consideration of attribute dependency graphs to ensure correct evaluation order without conflicts. Furthermore, representation choice affects compiler performance. Finally, selecting the right abstraction improves overall efficiency.

Q2. A compiler designer must choose between bottom-up and top-down evaluation strategies for implementing syntax-directed definitions. Justify this choice by analyzing grammar restrictions, attribute dependencies, parsing efficiency, and the ease of integrating semantic actions within each approach. These require careful consideration of attribute dependency graphs to ensure correct evaluation order without conflicts. Furthermore, evaluation strategy influences implementation complexity. Finally, choosing the right method ensures better performance.

Q3. In complex arithmetic and logical expressions involving multiple operators, precedence rules, and associativity constraints, attribute dependencies may create conflicts in evaluation order. Analyze how these dependencies affect semantic rule execution and propose techniques such as dependency graphs or evaluation scheduling to resolve them. These require careful consideration of attribute dependency graphs to ensure correct evaluation order without conflicts. Furthermore, resolving conflicts is critical for correctness. Finally, structured evaluation improves reliability.

Q4. A statically typed programming language enforces type checking during compilation to ensure correctness. Evaluate how such a type system helps detect errors early, prevents invalid operations, and improves software reliability, while also discussing any limitations in flexibility compared to dynamically typed systems. These require careful consideration of attribute dependency graphs to ensure correct evaluation order without conflicts. Furthermore, strong typing enhances safety. Finally, balancing flexibility and strictness is important.

Q5. A modern programming language supports advanced features such as polymorphism, function overloading, and type inference. Analyze the challenges faced by the compiler's type system in resolving types accurately, managing ambiguity, and maintaining efficient compilation performance. These require careful consideration of attribute dependency graphs to ensure correct evaluation order without conflicts. Furthermore, advanced features increase complexity. Finally, efficient resolution mechanisms are essential.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

Q1. Parse Tree vs Syntax Tree:

Introduction:

Both parse trees and syntax trees represent the structure of a program during compilation. However, a **parse tree** represents the complete grammatical structure produced by the parser, whereas a **syntax tree (AST)** represents the essential logical structure of the program with unnecessary grammar details removed.

Comparison:

Feature	Parse Tree	Syntax Tree / AST
Structure	Contains all grammar symbols and productions	Contains only essential program constructs
Memory Usage	High because it contains many nodes	Lower because redundant nodes are removed
Traversal	More complex and time-consuming	Easier and faster
Optimization	Less suitable directly	Highly suitable for optimization
Semantic Analysis	Can be used, but contains unnecessary details	Preferred because meaningful constructs are directly represented
Code Generation	Less convenient	Very convenient
Purpose	Shows how input follows grammar	Shows the actual logical structure of the program

Example:

For the expression:

$a + b * c$

A parse tree contains grammar elements such as **E**, **T**, and **F**, depending on the grammar. The syntax tree simplifies this to:

```

  +
 / \
a  *
   / \
  b  c
```

The AST clearly shows that multiplication has higher precedence than addition.

Analysis:

Parse trees are useful during **syntax analysis** because they demonstrate how the input string is derived from the grammar. However, they consume more memory and require more traversal.

ASTs remove unnecessary grammar information. Therefore, they are more efficient for **semantic analysis, intermediate-code generation, optimization, and final code generation**.

The choice of representation directly affects compiler performance. For a large program, using an AST reduces memory consumption and makes later compiler phases faster.

Conclusion:

A parse tree is best for representing the **complete grammatical derivation**, while an AST is better for representing the **meaningful program structure**. Therefore, modern compilers generally use ASTs after parsing because they provide efficient traversal, optimization, semantic analysis, and code generation.

Q2. Bottom-Up vs Top-Down Evaluation of SDD:

Introduction:

A **Syntax-Directed Definition (SDD)** associates attributes and semantic rules with grammar productions. The compiler must evaluate these attributes in an order that satisfies their dependencies. The two major approaches are **top-down** and **bottom-up** evaluation.

Bottom-Up Evaluation:

Bottom-up evaluation starts from the input symbols and moves toward the start symbol.

It is closely associated with **bottom-up parsing**, such as LR parsing.

Advantages:

- Works naturally with synthesized attributes.
- Semantic actions can be executed when a production is reduced.
- Suitable for LR parsers.
- Efficient for expression evaluation and intermediate-code generation.

Limitation:

- Inherited attributes may require special handling because information can need to flow from parent or sibling nodes.

Top-Down Evaluation:

Top-down evaluation starts from the start symbol and proceeds toward the input symbols.

It is commonly associated with **LL and recursive-descent parsing**.

Advantages:

- Naturally supports information flowing from parent to child.
- Convenient for inherited attributes.
- Semantic actions can be integrated while expanding productions.
- Easy to understand and implement for suitable grammars.

Limitations:

- Left-recursive grammars are problematic for recursive-descent parsing.
- Grammar transformation may be required.
- Attribute dependencies must still be carefully managed.

Comparison:

Factor	Bottom-Up	Top-Down
Direction	Leaves $\rightarrow$ Root	Root $\rightarrow$ Leaves
Common parsing	LR	LL
Synthesized attributes	Very suitable	Suitable
Inherited attributes	More difficult	More natural
Left recursion	Supported by LR methods	Must usually be removed
Semantic actions	During reduction	During expansion
Implementation	Efficient but may be complex	Simple for suitable grammars

Analysis:

The choice depends mainly on the **grammar and attribute dependencies**. If the SDD mainly contains synthesized attributes, bottom-up evaluation is generally preferable because attributes can be calculated during reductions.

If inherited information must flow from a parent to its children, a top-down strategy can be more natural.

Dependency graphs are important because they identify which attributes must be evaluated before others. An incorrect evaluation order can result in incorrect semantic information.

Conclusion:

There is no universally best strategy. **Bottom-up evaluation** is efficient for synthesized attributes and LR parsing, while **top-down evaluation** is convenient when inherited information flows through the grammar. The compiler designer should select the method based on grammar restrictions, dependencies, parsing efficiency, and implementation complexity.

Q3. Attribute Dependencies in Arithmetic and Logical Expressions

Introduction:

Complex expressions may contain several operators with different **precedence and associativity**. Syntax-directed semantic rules use attributes to calculate values or generate intermediate code. Dependencies between these attributes must be evaluated in the correct order.

Example:

Consider:

$a + b * c$

The multiplication must be evaluated before addition because $*$ has higher precedence.

The dependency can be represented as:

$$\begin{array}{c} + \\ / \backslash \\ a \quad * \\ / \backslash \\ b \quad c \end{array}$$

The semantic dependency is:

$b, c \rightarrow (b * c) \rightarrow a + (b * c)$

Therefore, the attribute for $b * c$ must be available before the attribute for the complete expression is calculated.

Problems Caused by Dependencies:

1. **Wrong evaluation order:**
An attribute may be evaluated before the values it depends on are available.
2. **Circular dependencies:**
An attribute may indirectly depend on itself.
3. **Precedence conflicts:**
Operators may have different precedence levels.
4. **Associativity conflicts:**
For example, subtraction is normally left associative:

$$a - b - c = (a-b)-c$$

5. **Logical expressions:**
Boolean expressions may require short-circuit evaluation, making execution order important.

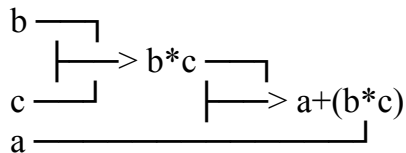
Dependency Graph:

A **dependency graph** represents relationships between attributes.

For:

$$a + b * c$$

the graph can be viewed as:



This graph determines a valid evaluation order.

Techniques to Resolve Conflicts:

1. Dependency Graphs:

Create a graph showing which attributes depend on other attributes.

2. Topological Ordering:

If the dependency graph is acyclic, perform a topological ordering to obtain a valid evaluation sequence.

3. Separate Precedence Levels:

Design grammar productions that explicitly represent operator precedence.

4. Associativity Rules:

Use grammar structure or semantic rules to enforce left or right associativity.

5. Evaluation Scheduling:

Schedule semantic actions only after their required attributes have been computed.

6. Detect Circular Dependencies:

Check the dependency graph for cycles before evaluation.

Conclusion:

Attribute dependencies directly affect the correctness of semantic-rule execution. Dependency graphs and evaluation scheduling provide systematic ways to determine the correct order. Proper handling of precedence, associativity, and dependencies ensures reliable expression evaluation and correct intermediate-code generation.

Q4. Static Type Checking

Introduction:

A **statically typed programming language** performs type checking during compilation. The compiler verifies that operations are performed only on compatible data types.

For example:

```
int a;
```

```
float b;
```

```
a = b;
```

The compiler can identify whether this assignment is valid according to the language's type-conversion rules.

How Static Type Checking Helps:

1. Early Error Detection:

Type errors are detected before the program is executed.

Examples include:

- Adding incompatible types.
- Assigning an incompatible value.
- Calling a function with incorrect argument types.
- Applying an operation to an inappropriate data type.

2. Prevents Invalid Operations:

The compiler can ensure that operations are applied to valid operands.

For example:

`int + int`

is valid, whereas an operation involving incompatible types may be rejected.

3. Improves Reliability:

Since many errors are identified during compilation, fewer type-related failures occur during execution.

4. Supports Better Optimization:

Knowing types at compile time allows the compiler to generate more efficient code and perform type-specific optimizations.

5. Improves Program Maintainability:

Explicit types make the intended use of variables clearer and allow compilers and development tools to detect mistakes earlier.

Limitations Compared with Dynamic Typing:

Static typing can reduce flexibility because the programmer must satisfy compile-time type restrictions.

Dynamic languages determine types during execution, which can make rapid development and experimentation easier. However, some errors may remain undetected until the relevant code is executed.

Comparison:

Static Typing	Dynamic Typing
Checking mainly at compile time	Checking mainly at runtime
Errors detected early	Some errors appear during execution
More predictable	More flexible

Can support compile-time optimization

May require runtime type checks

Strong compile-time guarantees

Greater flexibility

Analysis:

The major advantage of static typing is that it moves error detection earlier in the software development process. This improves reliability and can also improve compiler optimization.

However, excessive strictness can reduce flexibility. Therefore, modern languages often provide features such as type inference and controlled implicit conversions to balance safety with convenience.

Conclusion:

Static type checking provides **early error detection, safer operations, improved reliability, and optimization opportunities**. Although it can be less flexible than dynamic typing, its advantages make it highly valuable for developing reliable and maintainable software.